WACHSAM

Ein kamerabasierter Prototyp zur Erkennung längeren Augenschlusses

Technischer Projektbericht

Yaman Kisanieh

Stand: 3. November 2026

Kurzfassung

Wachsam ist ein studentisches Computer-Vision-Projekt für Windows. Eine einzelne Kamera liefert Bilder, aus denen ein vortrainierter MediaPipe Face Landmarker Gesichtspunkte bestimmt. Eigene geometrische Berechnungen schätzen die Augenöffnung und die Kopfrotation. Liegen die Messwerte beider Augen kontinuierlich mindestens eine Sekunde unter dem eingestellten Schwellwert, erzeugt das Programm einen visuellen und akustischen Alarm.

Die Oberfläche ermöglicht Start, Stopp, Kopfkalibrierung und Tontest. Messwerte werden als CSV protokolliert; Bilder und Videos werden durch die Anwendung nicht gespeichert. Die Anwendung wurde mit PyInstaller als Windows-Ordnerpaket gebaut.

Abgrenzung: Der Prototyp erkennt eine regelbasiert definierte Augenschlussbedingung. Er stellt weder tatsächlichen Schlaf noch die Fahrtüchtigkeit fest. Bei starker Kopfneigung, seitlicher Drehung oder verdeckten Augen kann die Messung ausfallen. Ein unabhängiger Kopfalarm wurde nach Entwicklungstests in der finalen Variante deaktiviert.

Projektart	Experimenteller Portfolio-Prototyp
Eingang	Eine Kamera, stationärer Laptop-Versuchsaufbau
ML-Anteil	Vortrainierter Landmark-Detektor, kein eigenes Training
Eigene Verarbeitung	EAR, Zeitlogik, Kopfrotation, Messprüfung, GUI und Protokolle
Validierungsstand	Entwicklungsversuche; keine belastbare Genauigkeitsstudie

1 Ziel und technischer Aufbau

Ziel war die Entwicklung einer lokal ausführbaren Anwendung, die mögliche Hinweise auf Müdigkeit über länger anhaltenden Augenschluss sichtbar macht. Der Schwerpunkt liegt auf der Integration eines vorhandenen ML-Modells, nachvollziehbarer Entscheidungslogik und einer bedienbaren Oberfläche.

Die Trennung in `main.py` und `app.py` vermeidet die Vermischung von Messfunktionen und Benutzeroberfläche. In der finalen Struktur enthält `main.py` gemeinsame Funktionen, Parameter und Dateipfade. `app.py` steuert Kamera, Verarbeitung, Oberfläche und Sitzungsprotokolle.

Technik	Aufgabe im Projekt
Python 3.12	Implementierung; verwendete Entwicklungsumgebung unter Windows
OpenCV	Kamera über DirectShow, Bildverarbeitung und BGR–RGB-Umwandlung
MediaPipe Tasks	Vortrainierter Face Landmarker, ein Gesicht, VIDEO-Modus, Gesichtspunkte und Transformationsmatrix [1]
NumPy / Mathematik	Punktabstände, SVD und Rotationsberechnungen
Tkinter / Pillow	Desktop-Oberfläche und skaliertes Livebild
Threading / Queues	Kamera-Worker getrennt von Tkinter; begrenzte Warteschlange für aktuelle Anzeige
winsound	Windows-Warnton; Tonwiedergabe in eigenem Thread
CSV / pathlib / time	Sitzungsdateien, Ressourcenpfade und monotone Zeitmessung
Matplotlib	Entwicklungsanalyse von EAR-Verläufen und Alarmphasen
PyInstaller	Verpackung als EXE mit unterstützenden Dateien [3]

Verarbeitung eines Kamerabildes

Nach dem Einlesen wird das Bild in RGB umgewandelt und an den Face Landmarker übergeben. Jeder Aufruf erhält einen strikt steigenden Millisekunden-Zeitstempel. Anschließend werden Augenpunkte und Kopfrotation ausgewertet. Erst bei geeigneten Messbedingungen wird der Schließzustand zeitlich verfolgt. Oberfläche und CSV erhalten Messwerte und Zustände.

Das neuronale Modell lokalisiert Gesichtspunkte; die eigentliche Alarmentscheidung stammt aus eigenen Regeln. Es wurden weder ein eigener Müdigkeitsklassifikator noch ein SVM trainiert. Gesichtsausdrucks-Scores, PERCLOS, eine validierte Blinzelfrequenzanalyse und Gähnerkennung gehören nicht zur finalen Implementierung.

2 Messverfahren und Entscheidungslogik

Eye Aspect Ratio (EAR)

Für jedes Auge werden sechs Punkte in Bildkoordinaten umgerechnet. Aus zwei vertikalen und einem horizontalen Abstand ergibt sich:

$$\mathrm{EAR} = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|}. \tag{1}$$

Die geometrische EAR-Grundidee ist in der Arbeit von Soukupová und Čech beschrieben [2]. Hier wird sie mit MediaPipe-Punkten und einer einfachen Zeitregel kombiniert; der im Original vorgestellte SVM-Klassifikator wird nicht übernommen.

Die verwendeten Indexgruppen lauten `[33,160,158,133,153,144]` und `[362,385,387,263,373,380]`. Die internen Bezeichnungen L/R werden aus dem Code übernommen und sind keine unabhängig verifizierte anatomische Seitenzuordnung.

Die Schließbedingung verlangt beide Werte unter **0,20**. Der Mittelwert dient der Anzeige und Diagnose, nicht allein der Entscheidung. Nach **1,0 s** durchgängiger gültiger Schließbedingung wird der Alarm aktiv. Ein kurzes Blinzeln erreicht diese Dauer typischerweise im Test nicht; das System klassifiziert jedoch keine klinischen Blinkmuster. Der EAR-Schwellwert wurde manuell erprobt und ist nicht für alle Personen validiert.

Kopfrotation und Kalibrierung

Aus dem oberen 3×3-Block der MediaPipe-Transformationsmatrix wird mittels Singulärwertzerlegung eine Rotationsmatrix angenähert. Die Kopfkalibrierung sammelt mindestens 20 Matrizen über mindestens zwei Sekunden. Der Mittelwert wird erneut auf eine Rotation projiziert; eine maximale Abweichung von fünf Grad dient als experimentelle Stabilitätsprüfung. Bei Messausfall oder größerer Bewegung beginnt die Sammlung erneut.

Relative Winkel werden aus RR_0^{T} abgeleitet: Pitch für Auf/Ab, Yaw für seitliche Drehung und Roll für Neigung zur Schulter. Diese Winkel sind Schätzwerte für die Softwarelogik und keine vermessenen Referenzwinkel. Die Kalibrierung passt die Kopfneutralstellung an, nicht die Augen-Schwelle.

Ungültige Messung und Rücksetzen

Die finale Variante prüft rohe und relative Winkel gegen Pitch $\pm 15°$, Yaw $\pm 20°$ und Roll $\pm 15°$. Außerdem werden nichtendliche und außerhalb des Bildes liegende Augenpunkte, horizontale Augenabstände unter acht Pixeln sowie EAR-Werte außerhalb $[0; 0{,}8]$ verworfen. Diese Heuristiken garantieren keine tatsächliche Sichtbarkeit des Auges.

Bei ungültiger Messung, fehlendem Gesicht oder Bildabständen über 0,5 s wird der Schließzähler zurückgesetzt. Fehlende Messwerte gelten weder als offene noch als geschlossene Augen. Kopfbewegung allein löst in der finalen Variante keinen Warnton aus.

3 Oberfläche, Ausführung und Protokollierung

Die dunkle Oberfläche verwendet Schwarz, Weiß und Blautöne. Start, Stopp, Kalibrierung und Tontest bleiben außerhalb des begrenzten Kamerabereichs erreichbar. Das Livebild wird in seine verfügbare Fläche skaliert, um das zuvor beobachtete Verdrängen der Bedienelemente zu vermeiden.

Der Kamera-Worker veröffentlicht aktuelle Ergebnisse über eine begrenzte Queue. Tkinter wird ausschließlich im GUI-Thread aktualisiert. Die Tonfunktion verwendet ein Lock gegen überlappende Tonfolgen; ein manueller Tontest prüft die Audioausgabe, nicht den Erkennungsalgorithmus. Ein Alarm wird visuell hervorgehoben und wiederholt akustisch ausgegeben, solange die gültige Schließbedingung erfüllt ist.

Zustandsgruppe	Bedeutung und Verhalten
Offen / geschlossen	Gültige Augenmessung; Zeitmessung nur bei beidseitiger Schließbedingung
Alarm	Zeitgrenze erreicht; Warnmeldung und Ton
Kalibrierung	Neutralstellung fehlt oder wird gesammelt; kein Augenalarm
Messung nicht verfügbar	Gesicht, Kopfrotation oder Augengeometrie nicht ausreichend; Zähler zurückgesetzt

Windows-Paket

Die Standalone-Version entsteht aus `app.py` mit `--onedir`, `--windowed`, `--collect-allmediapipe` und dem beigefügten Modell. Das Ergebnis ist `dist/Wachsam/Wachsam.exe` mit Begleitdateien. Der gesamte Ordner muss erhalten bleiben. Ein erfolgreicher Build belegt nur die Paketierung; Kamera, Modell, Ton und Protokolle der EXE müssen separat geprüft werden.

Ressourcen werden relativ zu `__file__` gesucht. Im eingefrorenen Programm werden Logs unter `%LOCALAPPDATA%/Wachsam/logs` gespeichert; beim Quellcode-Start liegen sie im Projektordner. Dieses Trennen von Ressourcen und Ausgabedaten berücksichtigt die veränderten Laufzeitpfade nach dem Verpacken [4].

CSV als Entwicklungsnachweis

Pro Sitzung wird eine neue CSV angelegt. Sie enthält EAR je Auge, mittleren EAR, Schließdauer, Messzustand, Alarmflag, rohe und relative Winkel, Kalibrierungsnummer sowie Parameter. Rohwerte können auch bei verworfenen Messungen vorhanden sein und dürfen dann nicht als gültige Entscheidung gelesen werden.

Die Protokolle dienen dem Nachvollziehen von Zeitlogik, Messausfällen und Fehlern. Sie enthalten keine Videoaufzeichnung und keine unabhängigen Labels für tatsächlichen Schlaf. Frühere Entwicklungsstände mit Kopfalarm werden getrennt archiviert.

4 Beobachtete Entwicklungsergebnisse

Für diesen Bericht wurden die zwei zuletzt bereitgestellten Dateien ausgewertet. Die Markdown-Kopie wurde durch Entfernen von Formatierungszeichen und Dekodieren von HTML-Zeichenreferenzen in CSV überführt. Herkunft und Hashwerte sind im beigefügten Manifest dokumentiert.

	GUI-Beispiel	Frühere Version
Messzeilen	193	1072
Zeitbereich	0,766–20,000 s	1,344–108,609 s
Augenschlussalarm	1 Ereignis / 13 Zeilen	1 Ereignis / 16 Zeilen
Erste Warnung	bei 1,032 s gemessenem Schluss	bei 1,047 s gemessenem Schluss
Kopfalarm	nicht im GUI-Schema	1 Ereignis / 71 Zeilen

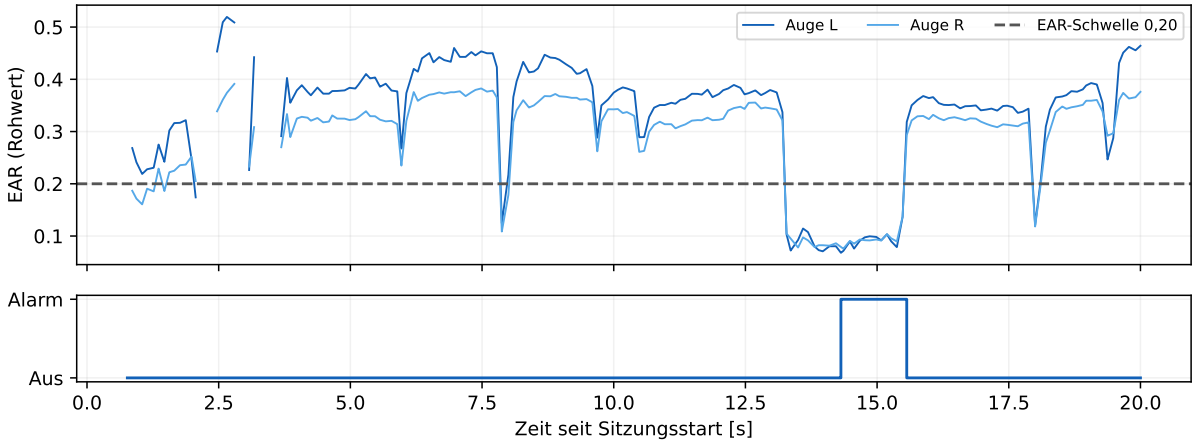


Abbildung 1: GUI-Beispielsitzung. EAR sind Rohwerte, auch vor Kalibrierung. Die Alarmspur zeigt die interne Entscheidung, keine extern annotierte Müdigkeit.

Im GUI-Beispiel bilden 13 Alarmzeilen ein zusammenhängendes Ereignis von 14,313 bis 15,484 s. Es wurden 47 OPEN-, 11 CLOSED-, 87 NOT_CALIBRATED-, 20 CALIBRATING-, zehn NO_FACE- und fünf unsichere Kopfpositionszeilen registriert. Die fünf unsicheren Zeilen haben Schließdauer und Alarm jeweils null.

Die frühere Sitzung enthält einen unabhängigen Kopfalarm ab 99,172 s. Dieser erklärt, warum eine ältere Version auch mit offenen Augen bei Kopfbewegung warnen konnte. Alle Benutzerlabels sind dort UNLABELED; 604 Zeilen befinden sich in Kalibrierung. Diese Daten dürfen nicht als Test der final deaktivierten Kopfalarmfunktion interpretiert werden.

Keine Genauigkeitsbehauptung: Bildzeilen sind keine unabhängigen Testfälle. Ohne externe Annotation, vollständige Soll-Ereignisse und mehrere Personen lassen sich daraus weder Accuracy noch Precision oder Recall für Müdigkeit ableiten. Auch eine richtige Reaktion auf absichtlichen Augenschluss beweist keine Schlaferkennung.

5 Grenzen und begründete Weiterentwicklung

Bekannte Einschränkung: Kopf zur Schulter

Neigt eine Person den Kopf stark zur Schulter oder nach unten, können Augen verdeckt sein oder die Winkelprüfung kann die Messung verwerfen. Der Zähler wird dann zurückgesetzt. Dadurch ist eine **ausbleibende Warnung trotz tatsächlicher Schläfrigkeit möglich**. Umgekehrt kann schlechte Landmark-Lokalisierung innerhalb der zugelassenen Winkel falsche Schließwerte erzeugen. Eine einzelne Kamera liefert keinen zuverlässigen 360-Grad-Blick auf beide Augen.

Die konservative Messprüfung reduziert bestimmte Fehlinterpretationen, erzeugt aber zugleich Überwachungslücken. Das ist ein dokumentierter Zielkonflikt, keine vollständig gelöste Erkennungsaufgabe. Beleuchtung, Brillen, Abstand, Personenunterschiede und stärkere Drehungen wurden nicht systematisch validiert. Die Tests fanden im stationären Entwicklungsaufbau statt.

Ist mehr als eine Kamera unprofessionell?

Nein. Ob zusätzliche Kameras sinnvoll sind, hängt von Sichtabdeckung, Einbauort, Synchronisierung, Rechenaufwand und Kosten ab. Als Serienbeispiel beschreibt Volvo für den EX90 zwei Kameras zur Fahrerbeobachtung [5]. Das belegt eine mögliche Architektur, aber nicht die Eignung für diesen Prototyp.

Vier Kameras werden hier nicht als optimale Lösung vorgeschlagen. Zuerst sollten Position und Optik einer Kamera sowie kontrollierte Beleuchtung verbessert und gemessen werden. Eine zusätzliche Kameraperspektive oder geeignete Nahinfrarot-Hardware kann anschließend als Vergleichsaufbau untersucht werden. Zusätzliche Hardware wurde im Projekt nicht implementiert, da sie nicht verfügbar war.

Nächste Schritte in sinnvoller Reihenfolge

1. Strukturierte Tests mit unabhängigen Augen-/Kopf-Labels; Fehlalarme, verpasste Ereignisse, Messverfügbarkeit und Warnlatenz dokumentieren.
2. Individuelle EAR-Kalibrierung und robustere Messqualitätsprüfung untersuchen. Kopfneigung als Zusatzsignal bewerten, nicht automatisch als Müdigkeit deklarieren.
3. Mehrere Personen, Beleuchtungen, Brillen und Kamerapositionen getrennt testen; ungültige Messphasen ausdrücklich ausweisen.
4. Optional zusätzliche Sensorik vergleichen. Bei mehreren Kameras Entscheidungen zeitlich abstimmen und widersprüchliche Messungen behandeln.

Fazit

Wachsam zeigt eine vollständige Anwendungskette von einer Kamera über vortrainierte Gesichtspunkt-Erkennung und geometrische Zeitlogik bis zu Desktop-Oberfläche, Protokollierung und Windows-Paketierung. Der konkrete Nutzen des Projekts liegt im nachvollziehbaren Prototypen und seiner Fehleranalyse. Für eine verlässliche Nutzung während realer Fahrten ist die vorliegende Evaluation nicht ausreichend.

Quellen und Begleitdateien

Literatur

[1] Google AI Edge: Face landmark detection guide for Python. https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker/python. Abruf: 03.11.2026.

[2] T. Soukupová und J. Čech: Real-Time Eye Blink Detection using Facial Landmarks. Computer Vision Winter Workshop, 2016. https://cmp.felk.cvut.cz/ftp/articles/cech/Soukupova-CVWW-2016.pdf.

[3] PyInstaller: Using PyInstaller. https://pyinstaller.org/en/stable/usage.html. Abruf: 03.11.2026.

[4] PyInstaller: Run-time Information. https://pyinstaller.org/en/stable/runtime-information.html. Abruf: 03.11.2026.

[5] Volvo Cars: EX90 – Driver behavior detection, Online-Betriebsanleitung. https://www.volvocars.com/us/support/car/ex90/26w17/article/47d2c97fd33effd3c0a8cc3718c999b7-85596e53922f2e19c0a8cc42679c08ea-8664b2fa77a7e089c0a8296870d1a409/47d2c97fd33effd3c0a8cc3718c999b7-2d47f76eac587475c0a8b0c1648accba-8664b2fa77a7e089c0a8296870d1a409/. Abruf: 03.11.2026.

Praktische Nutzung der Begleitdateien

Datei	Verwendung
`README.md`	Im Code-Repository als Startseite: Installation, Bedienung, Paketierung und Grenzen
`LOGS.md`	Erklärung der Messfelder, Altversionen und Evaluationsregeln
`test_results_template.csv`	Auszufüllende Testübersicht; keine erfundenen Versuchsergebnisse
`data/session_gui.csv`	Aufbereitete GUI-Beispielsitzung
`data/session_legacy.csv`	Historischer Vergleich mit früherem Kopfalarm
`data/session_manifest.json`	Herkunft, Hashwerte und Zustandszählungen
`requirements.txt`	Direkte Abhängigkeiten; getestete Versionen lokal ergänzen

Das Paket enthält die überarbeiteten Anwendungsquellen, das Modell und historische Messdaten. 18 synthetische Tests prüfen Geometrie und Zeitlogik; sie belegen keine Müdigkeitserkennungsrate. Windows-Oberfläche, Kamera, Ton und ein neuer EXE-Build wurden bei dieser Überarbeitung nicht erneut ausgeführt. Die EXE muss unter Windows neu erstellt werden.